

Biocompiler 2.0

RNA Processor – Especificação da Atividade

Objetivo: receber um pré-mRNA e simular, de forma didática, sua maturação até a geração de um mRNA maduro.

1. Papel do BioCompiler 2.0

Entrada: *pré-mRNA* bruto produzido pelo BioCompiler 1.0.

Processamento: reconhecimento e validação de íntrons, splicing, representação da CAP 5' e adição da cauda poli-A.

Saída: *mRNA maduro*, quando a entrada for válida; caso contrário, um diagnóstico do problema.

Fase I: *DNA transcrito em pré-mRNA* [BioCompiler 1.0](#)

Fase II: *pré-mRNA amadurecido com mRNA maduro* [BioCompiler 2.0](#) (objeto desse trabalho)

Fase III: *mRNA traduzido em proteína* [Sr. Ribossomo](#) (trabalho futuro)

2. Formato de entrada

A entrada será um arquivo texto (.txt) contendo uma ou mais sequências de pré-mRNA. Cada linha do arquivo corresponde a uma sequência independente. O programa deverá processar as linhas na ordem em que aparecem no arquivo.

Exemplo

Nome: *arquivo entrada.txt*

```
CCUAUGGCUGUAACCUUUAACUAACAAGAUGGCCUAC
CCUAUGGCUCUAACCUUUAACCUAACAGGAUGGCCUAC
CCUAUGGCUGUCCUUUCCUCCUCCAGGAUGGCCUAC
```

3. Convenção didática para o splicing

Para tornar o problema implementável e corrigível automaticamente, o BioCompiler 2.0 usará uma gramática simplificada. Ela representa conceitos reais do splicing, mas não pretende reproduzir toda a complexidade biológica.

ÍNTRON VÁLIDO: GU ... A ... AG

Elemento	Representação	Função didática
Sítio 5'	GU	Marca o início do íntron.
Branch point	A	Adenina localizada dentro do íntron.
Sítio 3'	AG	Marca o final do íntron.

4. Regra do branch point

Para a atividade, considere válido um **branch point A** localizado entre 10 e 30 nucleotídeos antes do AG terminal do íntron. Essa é uma regra computacional da atividade, criada para que a validação seja objetiva.

5. Operação de splicing

Quando um íntron válido for reconhecido, o programa deverá remover todo o trecho compreendido entre o GU inicial e o AG terminal, incluindo GU e AG, e unir diretamente os éxons adjacentes.

ANTES: **EXON1** (GU ... **A** ... AG) **EXON2**

DEPOIS: **EXON1EXON2**

6. CAP 5'

Após o splicing bem-sucedido, o programa deverá representar a adição da CAP 5' acrescentando o marcador computacional **m7Gppp** ao início da sequência processada. Esse texto é uma representação didática da estrutura química da CAP, não uma sequência natural de RNA.

7. Cauda poli-A

Após o splicing, o programa deverá adicionar exatamente 100 adeninas (A) à extremidade 3'. O valor fixo elimina ambiguidades e facilita a correção automática. Na célula real, o comprimento da cauda poli-A é variável.

8. Casos que o programa deverá reconhecer

# Caso	Condição	Saída
1  Correto	Existe GU ... A ... AG, com branch point válido	CORRETO
2  Sítio 5' ausente	Existe AG, mas não existe GU anterior compatível	BUG - sítio 5' ausente
3  Sítio 3' ausente	Existe GU, mas não existe AG posterior compatível	BUG - sítio 3' ausente
4  Branch point	Existem GU e AG, mas não há A válido entre 10–30 nt antes do AG	BUG - branch point

9. Saída padrão para a tela

A saída exibida na tela será detalhada e destinada ao usuário. Para cada linha, o programa mostrará o número da entrada, o status, o diagnóstico e, quando aplicável, o mRNA maduro.

```
=====
BIOCOMPILER 2.0 - RNA PROCESSOR
=====
ENTRADA: 1
STATUS: CORRETO
Sítio 5': OK
Branch point: OK
Sítio 3': OK
Splicing: OK
CAP 5': ADICIONADA
Cauda poli-A: 100 A
mRNA MADURO: m7GpppSEQUÊNCIA_PROCESSADAcadaPOLI-A
-----
```

10. Processamento em lote

O arquivo poderá conter uma ou várias linhas. O programa deverá processar cada linha de forma independente, preservar a ordem das entradas e gerar um resultado correspondente para cada uma delas.

11. Saída padrão para arquivo exportado

Além da saída na tela, o programa deverá gerar um arquivo de resultados. Para facilitar a correção automática, recomenda-se um registro por linha, com campos separados por ponto e vírgula.

```
linha;status;resultado;mRNA_maduro
1;OK;CORRETO; m7GpppSEQUÊNCIA_PROCESSADAcadaPOLI-A
2;ERRO;BUG - sítio 5';NÃO GERADO
3;ERRO;BUG - branch point;NÃO GERADO
```

Padrão proposto: arquivo **resultados.txt** (*ou .csv, se desejado*), codificação UTF-8. Os textos da coluna 'resultado' devem ser reproduzidos exatamente como definidos na especificação.

12. Requisitos de implementação

- Linguagens aceitas: Java ou Python.
- O programa deve ler as entradas a partir de arquivo texto.
- Uma entrada por linha.
- Não assumir previamente qual caso está sendo analisado.
- Não utilizar separadores internos na sequência de entrada.
- O BioCompiler 2.0 termina no mRNA maduro; ele não realiza tradução de proteínas.

13. Resumo do algoritmo

```

LER linha
↓
VALIDAR caracteres
↓
LOCALIZAR e VALIDAR sinais de splicing
↓
REMOVER íntrons + UNIR éxons
↓
ADICIONAR m7Gppp na extremidade 5'
↓
ADICIONAR 100 A na extremidade 3'
↓
GERAR mRNA MADURO + RELATÓRIO
  
```

Princípio da atividade: o aluno deve implementar um parser molecular, não memorizar casos de teste.

14. Exemplos de entrada e saída por caso

# Caso	O que o algoritmo encontra	Exemplo	Saída
1  Correto	Encontra GU, depois branch point A válido e depois AG	CCC  GU CCCC  A CCCCCCCCCCC  AG CCC	CORRETO
 Íntron 2 incompleto – falta 5'	Encontra um AG, mas não existe GU anterior que possa iniciar o íntron	CCCCCCCCC  A CCCCCCCCCC  AG CCC	BUG - sítio 5' ausente
 Íntron 3 incompleto – falta 3'	Encontra um GU, mas não existe AG posterior que possa terminar o íntron	CCC  GU CCCC  A CCCCCCCCCCCCCCCC	BUG - sítio 3' ausente
4  Branch point	Encontra GU e AG, mas não há A válido entre 10–30 nt antes do AG	CCC  GU CCCCCCCCCCCCCCCCC  $\emptyset$ A  AG CCC	BUG - branch point